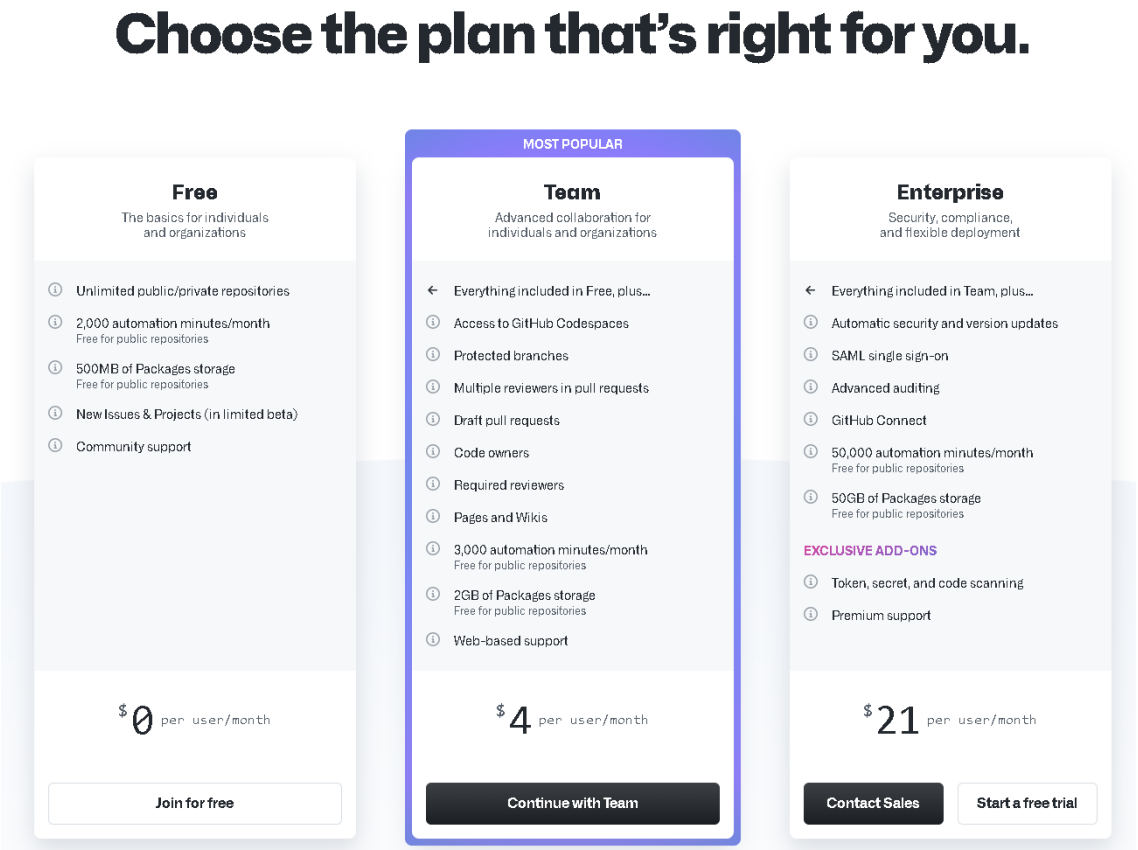
Git e GitHub

Visão Geral

O Git é uma ferramenta de **versionamento de código** ou controle de versão. Foi desenvolvida por Linus Torvalds, criador do sistema operacional Linux e um grande nome do movimento da Internet Livre que se desdobrou nos movimentos open source e open code. O proposito maior do Git é mantermos um histórico de alterações do nosso código, sendo mais fácil retornar para versões anteriores do mesmo, caso tenhamos algum problema em nossas novas versões e updates. Alguns termos como *push* (enviar o nosso código para o repositório), *pull request* (solicitar para que outra pessoa dê um pull no nosso repositório, pois não temos permissão para alterá-lo), *fork* (copiar um repositório de terceiro para sua conta), *clone* (copiar um repositório seu da nuvem para a máquina local) e *rollback* (retornar a versão anterior e estável do código) se tornaram mais populares a partir do uso do Git.

O Github é uma plataforma de hospedagem de códigos-fonte que utiliza o Git no seu controle de versões, portanto, está diretamente ligada ao mesmo mas é mais complexa que o mesmo. Com ele, vários programadores do mundo todo conseguem trabalhar em um mesmo código simultaneamente, eliminando a necessidade de ficar comparando códigos desenvolvidos em paralelo antes de lançar uma versão "oficial". Podemos usar o Github tanto para o desenvolvimento de códigos *open-source* como para repositórios privados e pessoais.

Github Free X Github Pago



Obs. Praticamente toda a empresa séria de TI usa o GitHub pago, mas para fins de portfólio o Free atende perfeitamente. Modelo de negócios esperto!

Github Pages

O GitHub Pages é um serviço de hospedagem de site estático que usa arquivos HTML, CSS e JavaScript diretamente de um repositório no GitHub, executa os arquivos por meio de um processo e publica um site. Você pode ver exemplos de sites do GitHub Pages na coleção de exemplos do GitHub Pages (https://github.com/collections/github-pages-examples).

Concorrência

HEROKU GitLab netlify

Instalação

1. Entrar no site: https://git-scm.com/ e fazer o download e a instalação do Git.
2. Após a instalação, criar uma pasta teste no diretório de sua preferência.
3. inicializar o Gitbash. Para isso, botão direito do mouse e GitBash Here. O Gitbash é o terminal do Git, uma espécie de "prompt de comando" que utilizamos para fazer todas as nossas operações.
4. Testar se a instalação correu bem utilizando o comando "git --version" no Gitbash.
5. Se tudo deu certo, cadastre-se no GitHub (https://github.com/).
6. Crie um repositório no GitHub e observe as instruções (tive um problema devido à mudança de nome de "master" para "main", mas eles descreviam isso no site).

Configurações Iniciais

7. Vamos fazer a configuração global do seu usuário. Essa etapa é importante porque vai gerar a sua *assinatura digital*. Desta forma, todas as mudanças de código que você fizer em um repositório serão identificadas como de sua autoria para os outros desenvolvedores. Seguir os seguintes passos:

    a. Configurar o nome com: git config --global user.name "Seu Nome Aqui".
    b. Configurar o seu e-mail com: git config –global user.email user@email.com

8. Agora vamos ligar o nosso repositório local ao GitHub. Usar o comando:
   git remote add origin https://github.com/nome_usuario/nome_repositorio.git

Criando Diretórios e Arquivos

9. Agora podemos partir para a criação do nosso primeiro repositório! =]

    a. Criar um novo diretório para o seu projeto com: mkdir nome_projeto
    b. Acessar o diretório com: cd nome_projeto/
    c. Inicializar o seu repositório com: git init

    Dica: se habilitar a visualização de arquivos ocultos do seu Windows, você verá que uma pasta .git foi criada. Essa pasta salva todas as alterações que você fizer localmente no seu repositório e auxilia no processo de subir as alterações locais para o Github.

10. Vamos agora criar arquivos dentro do diretório que está nosso projeto.

    a. Você pode criar um arquivo de texto simples utilizando: touch filename.txt

b. O GitBash tem integrado a ele o editor de textos básico do Linux (tipo o bloco de notas) chamado Vim. Para abrir o arquivo que acabamos de criar, podemos usar: vim filename.txt
c. Digitar algum texto de teste no arquivo. Depois :w para salvar o que foi digitado e :x para sair.

Dica: Você poderia fazer o mesmo processo com arquivos com outras extensões, como, por exemplo, arquivo ".py" ou ".ipynb". Mas é mais fácil desenvolvê-los em outros lugares e simplesmente arrastá-los depois para a pasta criada.

Monitorando nosso repositório

11. Indicar que o arquivo criado deve ser monitorado pelo controle de versões (**tracked**), utilizando git add.

    a. Se quiser adicionar todos os arquivos da pasta, use: git add .
    b. Se quiser adicionar algum arquivo específico, use: git nome_arquivo.extensao
    c. Para verificar todos os arquivos do projeto (adicionados ou não no sistema de monitoramento), utilizar git status.

    Dica: Crie um arquivo novo utilizando o procedimento do passo 7 e veja como ele aparece quando não é adicionado no monitoramento do repositório.

O primeiro Commit

12. Todos os arquivos que adicionarmos no passo 8 irão compor a versão do nosso código que será enviada ao repositório, ou em Programadorês "**commitada**". Localmente, chamamos esse conjunto de arquivos na nossa máquina que serão upados de **área de estágio**. Vamos fazer o primeiro commit por meio do comando: git commit -m "Informações sobre a versão que estou upando para que os outros programadores saibam o que eu estou fazendo".

13. Ao usar git status veremos que nossa área de estágio está limpa, pois o conjunto de arquivos locais corresponde exatamente a aquilo que está no repositório. Tudo o que modificarmos daqui para a frente fará parte de uma nova versão do projeto.

Versionamento

14. Se modificarmos nome_arquivo.txt, podemos ver as alterações feitas em relação à versão anterior utilizando o comando git diff.

15. Vamos fazer o nosso segundo commit. Para isso, alterar o arquivo, adiciona-lo no sistema de monitoramento e commitá-lo.

16. Agora podemos acessar nosso histórico de versões utilizando git log. Veja que cada commit recebe um ID único, também conhecido por **hash**.

Fork, Pull Request e Outros

Só aprendemos a enviar coisas para o nosso próprio repositório, que é basicamente tudo o que precisamos para montar o nosso portfólio. Contudo, existem vários outros conceitos importantes que envolvem a noção de **branches**, **níveis de permissão**, **processo de code review, etc.** Por isso, vamos assistir juntos o vídeo: https://www.youtube.com/watch?v=Du04jBWrv4A.

Após assistir, podemos tentar fazer o nosso primeiro fork, pull request, etc.

Obs. Videoaula didática, para se aprofundar: https://www.youtube.com/watch?v=2alg7MQ6_sI